Technical Report: Multi-Object Detection and Persistent ID Tracking in Sports Broadcast Footage

Project: Sports Player Tracking Pipeline
Model: YOLOv11x + BoT-SORT / ByteTrack
Sport: Cricket (IPL Broadcast)
Video Duration: 85.9 seconds | 2,147 frames | 1920×1080 @ 25 FPS
Date: April 2026

1. Model / Detector Used

Detector: YOLOv11x (Ultralytics, 2024)

YOLOv11x is the largest and most accurate variant in the YOLOv11 family, offering state-of-the-art object detection performance. It was selected for this task due to:

- **Superior detection accuracy** at the cost of moderate inference speed — the right tradeoff for offline video processing where accuracy matters more than real-time throughput
- **Native `ultralytics` integration** with built-in tracker support, enabling seamless pipeline construction
- **Pre-trained on COCO** with robust generalisation to person detection across varied scales, lighting, and occlusion scenarios
- **CUDA acceleration** — on the RTX 4060 GPU used in this project, `yolo11x` achieves 15–17 FPS on full 1080p frames, processing the entire 86-second video in under 3 minutes

The model was run in **person-only mode** (`classes=[0]`), filtering all non-human detections before passing results to the tracker. Detection confidence threshold was set to **0.35** after empirical tuning (see Section 6 — Challenges).

Variant	Parameters	mAP (COCO)	Use Case
YOLOv11n	2.6M	39.5	Fast prototyping
YOLOv11s	9.4M	47.0	Balanced
YOLOv11x	56.9M	54.7	Production (this project)

2. Tracking Algorithm Used

Two trackers were evaluated in parallel to enable direct comparison:

2.1 BoT-SORT (Primary Tracker)

BoT-SORT (Bootstrap Object Tracking — SORT) combines IoU-based motion prediction with camera motion compensation (CMC) via sparse optical flow. Key components:

- **Kalman Filter** for state prediction (position, velocity) between frames
- **Hungarian Algorithm** for optimal detection-to-track assignment
- **GMC (Generalised Motion Compensation)** using `sparseOptFlow` — compensates for camera pan and zoom, which are constant in broadcast cricket footage
- **Two-stage association** — high-confidence detections matched first (threshold 0.40), then low-confidence detections matched against remaining tracks (threshold 0.08)
- **Track buffer of 60 frames** (2.4 seconds) — keeps lost tracks alive across brief occlusions and cut transitions before reassigning a new ID

2.2 ByteTrack (Comparison Tracker)

ByteTrack improves on SORT by using **every detection** (including low-confidence ones) in a two-stage matching process, rather than discarding uncertain detections. This is particularly effective for partially occluded players where confidence scores are naturally lower.

- No camera motion compensation (unlike BoT-SORT)
- Simpler and faster — achieves 30.4 FPS vs. 16.8 FPS for BoT-SORT
- Slightly higher ID switch count due to the absence of CMC

3. Why This Combination Was Selected

The YOLOv11x + BoT-SORT combination was chosen as the primary pipeline for the following reasons:

YOLOv11x over smaller variants: The nano model (YOLOv11n) was tested first and produced 93 unique IDs on a skip-5 test run, demonstrating significant false-positive noise at broadcast scale. The extra model capacity of YOLOv11x dramatically reduces ghost detections and correctly sizes bounding boxes on distant players.

BoT-SORT over ByteTrack for cricket: Cricket broadcast footage involves constant camera motion — panning across the field, zooming on the batsman, cutting between end-on and side-on views. BoT-SORT's optical flow-based camera motion compensation (`gmc_method: sparseOptFlow`) accounts for frame-to-frame camera displacement before computing IoU distances, which is the correct approach for this domain. ByteTrack does not compensate for camera motion and consequently produces more ID switches in panned sequences.

ByteTrack retained as comparison: ByteTrack's speed advantage (30.4 FPS vs. 16.8 FPS) and its two-stage low-confidence detection strategy make it a strong candidate for real-time deployment scenarios where camera motion is limited. Including it provides a meaningful model comparison required by the assignment.

4. How ID Consistency Is Maintained

Persistent ID assignment is maintained through a multi-layered strategy:

4.1 Kalman Filter Prediction

Each active track maintains a Kalman filter state `[x, y, w, h, vx, vy, vw, vh]`. When a player is not detected in a frame (due to brief occlusion or detector miss), the Kalman filter **predicts** the track's next position using its last known velocity. This allows the track to remain alive and re-associate with the player when they reappear, without spawning a new ID.

4.2 Camera Motion Compensation

Sparse optical flow (`sparseOptFlow`) computes a global affine transform between consecutive frames by tracking feature points in the background (grass, pitch markings). This transform is subtracted from all track positions before IoU matching, ensuring that camera pan does not cause artificial displacement that would break track associations.

4.3 Two-Stage Hungarian Matching

- **Stage 1:** High-confidence detections (conf ≥ 0.40) are matched to active tracks using the Hungarian algorithm with IoU distance. Match threshold set to 0.85.
- **Stage 2:** Remaining unmatched detections (conf ≥ 0.08) are matched against lost/tentative tracks, recovering players who were briefly occluded or under-detected.

4.4 Track Buffer (60 frames)

Tracks are not immediately terminated when a player leaves the frame or is occluded. A 60-frame buffer (2.4 seconds at 25 FPS) keeps the track in a "lost" state. If the player reappears within this window and the IoU match succeeds, the original ID is restored without a switch.

4.5 Minimum Bounding Box Area Filter

A minimum area threshold of 2,000 px^2 (approximately 45×45 pixels) is applied post-detection to suppress ghost detections from distant crowd members, advertising boards, and image compression artifacts. This significantly reduces spurious ID spawning.

5. Quantitative Results

Pipeline Comparison

Metric	BoT-SORT	ByteTrack
Total Unique IDs assigned	181	184
ID switches (approx.)	177	180
Avg. players detected per frame	2.07	2.04
Max players in single frame	11	10
Frames with zero detections	511 (23.8%)	528 (24.6%)
Processing speed (FPS)	16.8	30.4
Fastest detected player	39.82 km/h (ID 5)	39.29 km/h (ID 553)

Winner: BoT-SORT — fewer ID switches, better camera motion handling, at the cost of processing speed.

6. Challenges Faced

6.1 AV1 Codec Incompatibility

The source video downloaded from YouTube was encoded in **AV1 (libdav1d)**, which is not natively supported by OpenCV on Windows. The video was re-encoded to H.264 using FFmpeg (`libx264`, CRF 18) before pipeline processing. The NVENC hardware encoder (`h264_nvenc`) was also used for faster re-encoding at 20.8× real-time speed.

6.2 Broadcast Camera Dynamics

Cricket broadcast footage involves aggressive camera behaviour that fundamentally challenges tracking:

- **Hard cuts** between wide-field and close-up angles reset all active tracks, as there is no spatial continuity between frames
- **Camera pan and zoom** during play creates artificial displacement of all player positions simultaneously, mimicking rapid motion
- **Slow-motion replays** interleaved with live footage cause the same players to be re-detected with fresh IDs

These cut patterns explain the `avg_players_per_frame = 2.07` statistic. During active play frames, 8–11 players are detected correctly. During scoreboards, replays, and transitions, the count drops to zero — and these null frames compose approximately 24% of the broadcast.

6.3 Uniform Appearance Similarity

All fielders in cricket wear near-identical white uniforms. Appearance-based ReID (which distinguishes players by clothing colour and texture) offers minimal discriminative power in this scenario. This increases reliance on purely geometric (position + velocity) association, which is less robust when players cross paths or a camera cut repositions them to new screen coordinates.

6.4 Scale Variation

Players range from large (batsmen in the foreground, ~200×400 px) to tiny (deep fielders, ~25×60 px). The minimum bounding box filter (`MIN_AREA = 2000 px²`) was tuned to exclude genuine background noise while retaining the smallest on-field players. Detection confidence was set to 0.35 — lower than the default 0.5 — to catch distant fielders at the edge of the model's detection capability.

6.5 Confidence Threshold Tuning

Initial runs at `conf=0.45` produced 166 unique IDs with only 1.92 avg detections per frame. Reducing to `conf=0.35` improved average detections to 2.07 but the fundamental broadcast cut challenge remained the limiting factor, not the threshold.

7. Failure Cases Observed

Failure Case	Description	Impact
Hard camera cuts	Abrupt scene change between delivery and replay; all tracks terminate and restart	Major — accounts for ~80% of ID switches
Slow-motion replay	Same over replayed at 0.5× speed; players re-detected with new IDs	Moderate — adds ~15-20 spurious IDs per replay
Crowd spill-over	Spectators in the foreground or front rows occasionally detected as players	Minor — filtered by `MIN_AREA` in most cases
Umpire detection	Umpires in dark coats occasionally tracked alongside white-clad players	Minor — does not affect player ID consistency
Tight occlusion	Two players crossing paths at close range; bounding boxes merge then separate with swapped IDs	Rare — 3-4 observed instances in test video

8. Possible Improvements

8.1 Appearance-Based ReID with Jersey Number Recognition

Integrating an OSNet-based ReID model (via `boxmot`) or a dedicated **jersey number OCR** module would enable player re-identification across camera cuts by appearance rather than geometry alone. This is the single highest-impact improvement for cricket tracking.

8.2 Shot Boundary Detection

Detecting hard camera cuts using frame-difference energy metrics and **resetting the Kalman filter states** at boundaries (rather than letting the track buffer expire naturally) would prevent the tracker from attempting to match pre-cut and post-cut detections across incompatible viewpoints.

8.3 Homography-Based Bird's-Eye Projection

A manual or auto-estimated homography between the broadcast camera view and a canonical top-down cricket field model would allow tracking in a **stable coordinate space** independent of camera motion, eliminating the CMC requirement entirely.

8.4 Team Clustering

K-means or DBSCAN clustering on player bounding box colours could automatically separate batting team from fielding team, enabling team-aware tracking that reduces false cross-team associations.

8.5 Higher Track Buffer for Replay Tolerance

Increasing `track_buffer` from 60 to 125 frames (5 seconds) would allow tracks to survive across a standard slow-motion replay sequence (~3-4 seconds), potentially recovering the same ID when live play resumes.

9. Technology Stack

Component	Technology	Version
Detector	YOLOv11x	Ultralytics 8.x
Primary tracker	BoT-SORT	Ultralytics built-in
Comparison tracker	ByteTrack	Ultralytics built-in
Video I/O	OpenCV	4.x
Re-encoding	FFmpeg 8.1	h264_nvenc / libx264
Heatmaps	Matplotlib + NumPy	3.x
Speed estimation	NumPy (pixel calibration)	—
Demo app	Streamlit	1.x
GPU	NVIDIA RTX 4060	CUDA 12.x
Language	Python	3.11